

Bringing Up Modern LLMs on Tenstorrent Blackhole

Aditya Sriram

Department of Computer Science, Stanford University adisram@stanford.edu

Project

End-to-end bringup of three modern LLM families on a single 1×4 **Blackhole P150** mesh: dense GQA, MoE with Gated DeltaNet recurrence, and a hybrid sliding/global-attention model. Every model walks from HF reference to a continuous-batching HTTP chat server, gated end-to-end against a CPU fp32 oracle.

Hardware

Mesh	1 × 4 Blackhole P150 (4 chips)
DRAM	32 GB GDDR6/chip, 404 GB/s measured stream
On-chip	110 Tensix cores, 1.39 MiB L1/core
Host link	1.4 GB/s tilize-bound

Models Brought Up

Model	Params	Architecture	Status
Qwen3.6-27B	27B	Dense GQA, TP4	shipped
Qwen3.6-35B-A3B	35B	MoE + Gated DeltaNet (~3B active)	shipped, $B=1$
Gemma 4 12B IT	12B	Hybrid sliding/global attention	shipped

All three reach byte-stable HTTP chat with continuous batching; each new model reused ~80–90% of the previous bringup’s scaffolding (CB engine, dev harness, HF oracle, paged SDPA).

Methodology

A staged ladder gates each rung on cosine vs. an HF fp32 oracle: single-layer → full forward → multi-step decode → long context → trace capture → continuous batching → HTTP. Validation is teacher-forced multi-step cosine *and* MAD per layer; six of seven silent-failure classes are invisible at decode pos 0 alone. Throughout, comparison is to ground truth, never a sibling TT path.

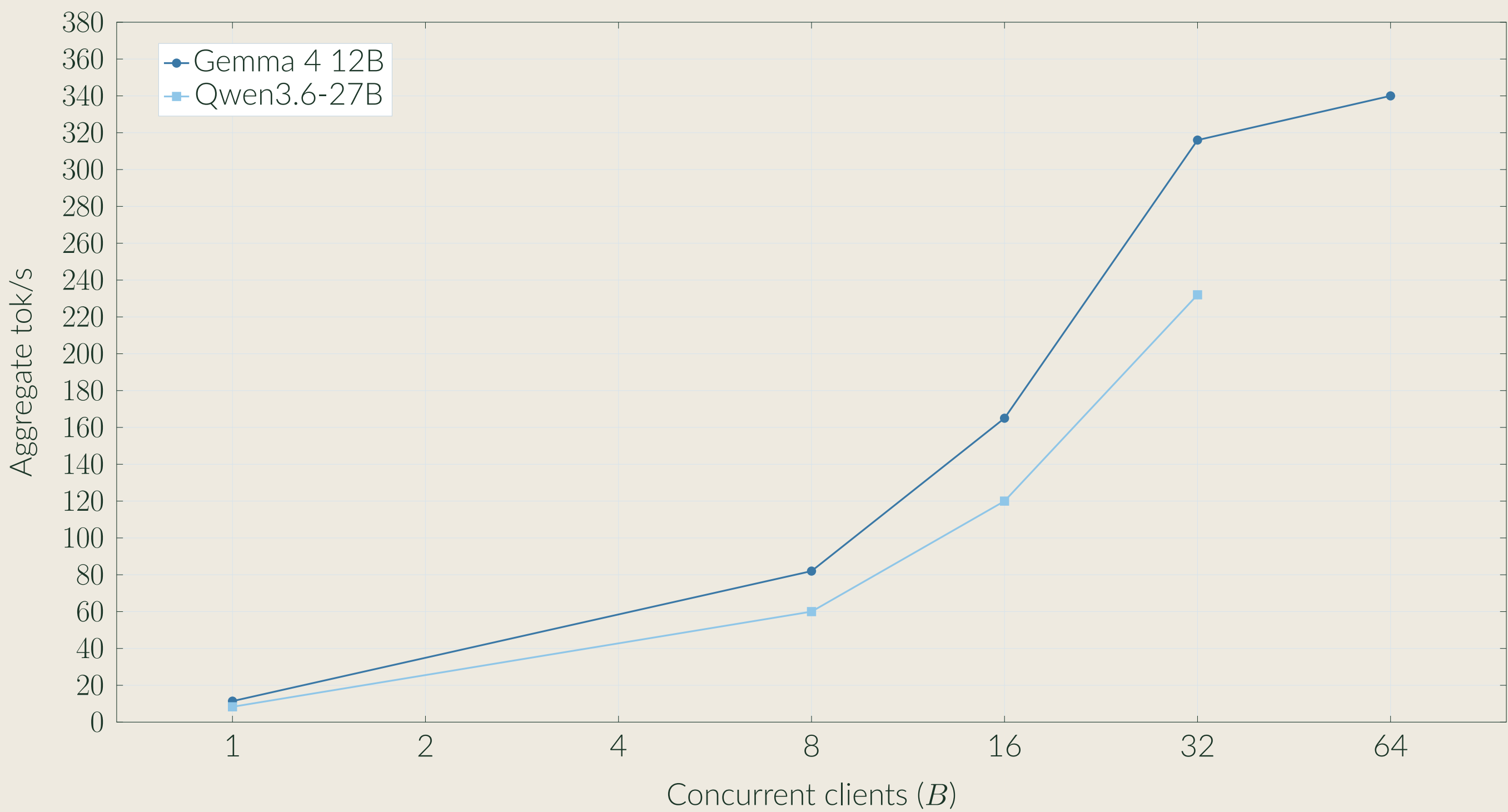
Headline

Gemma 4 12B: 316 tok/s aggregate at 32 concurrent clients.

Qwen3.6-27B: 232 tok/s aggregate at 32 concurrent clients.

28–29× near-linear scaling from $B=1$ via continuous batching.

Throughput vs. Concurrent Clients



Diminishing returns past $B=32$ on Gemma 4 (316 → 340 at $B=64$); scheduler + paged SDPA overhead saturate the weight-bandwidth amortization win.

Roofline

Decode at $B=1$ is weight-bandwidth-bound: $t_{\min} \approx \frac{\text{active GB/chip}}{404 \text{ GB/s}}$.

Model	Traced ms/tok	tok/s	% ceiling ($B=1$)
Gemma 4 12B	47.5	21.0	31% of 67 tok/s
Qwen3.6-27B (TP4)	77.0	12.9	22% of 60 tok/s
Qwen3.6-35B-A3B	81.2	12.3	4.6% of 270 tok/s

Continuous batching closes the gap on real workloads: $B=32$ amortizes weight reads $\sim 28\times$.

What Mattered

Three changes account for the majority of the throughput delta over a naive port.

- **Paged SDPA with HiFi2 fp16 accumulation** — kernel-time win, +62% on 27B TP single-stream.
- **Vocab-sharded lm_head with on-device argmax** — collapses a 600 KB host readback to 8 bytes; +8% on Gemma 4, +5% on 27B.
- **Two-phase trace warmup** — compile all paths first, then capture all traces back-to-back. Without it, decode JIT corrupts prefill trace memory at multi-trace coexistence (→ 99% CPU hang).

Correctness

Three orthogonal gates, run per model on every change.

Gate	27B	35B-A3B	Gemma 4
HF cosine ladder	PASS	$\cos_{L,32} \geq 0.987$	PASS
Multi-turn KV retention	PASS	3/3 PASS	PASS
Traced = Eager	PASS	PASS	100/100
Needle haystack	$L=500$	coherent	3/3 ($L \leq 512$)

Validation gates against a CPU fp32 oracle with per-sub-op hooks. Cosine alone is insufficient ($\cos(a, ka)=1$); MAD is reported at every probed position.

Continuous Batching

Iteration-level Orca-style scheduler: per-step admission and eviction, per-slot paged SDPA over a shared KV pool, slot-level content-keyed prefix cache. Each slot advances at its own position in a single batched forward.

Prefix Cache in Practice

Two-turn 27B chat over HTTP:

Turn	Prompt	Prefill
1 (cold)	105 tok	19.26 s
2 (PC hit)	172 tok	3.06 s

A prefix-cache hit on turn 2 yields a **6.3×** prefill speedup despite the longer prompt; the new prompt re-uses the cached KV for the shared prefix and only computes the suffix.

Conclusion

A single 1×4 Blackhole P150 mesh, paired with a disciplined bringup recipe, takes a modern LLM from HF spec to byte-stable HTTP chat at **316 tok/s aggregate** on 12B and **232 tok/s** on 27B. The hardware is competitive on real-workload throughput; the open software stack lets a small team bring up new architectures (MoE recurrence, hybrid attention, dual-head-dim) end-to-end without waiting on a vendor roadmap.